

Nhóm 12: Bổ sung lúc demo, show code

1. Logic chức năng

1.1 Dữ liệu gõ raw của client đi như thế nào?

Client gửi **TYPING_UPDATE** mỗi **300ms** lên server:

src/TypeRacer.Server/Handlers/RaceForm.cs

```
422     private async void UpdateTimer_Tick(object? sender, EventArgs e)
423     {
424         if (_isFinished) return;
425
426         try
427         {
428             await AppState.Instance.Client.SendAsync(MessageType.TYPING_UPDATE, new TypingUpdatePayload
429             {
430                 RoomCode = _roomCode,
431                 CurrentPosition = _typingDisplay.CurrentPosition,
432                 CorrectChars = _typingDisplay.CorrectCount,
433                 WrongChars = _typingDisplay.WrongCount,
434                 TypedText = _typingDisplay.TypedText,
435                 Timestamp = DateTimeOffset.UtcNow.ToUnixTimeMilliseconds(),
436             });
437         }
438         catch { }
439     }
```

Trong payload có:

- **CurrentPosition**
- **CorrectChars**
- **WrongChars**
- **TypedText**
- **Timestamp**

Server nhận dữ liệu này ở:

src/TypeRacer.Server/Handlers/GameHandler.cs

```

276 private async Task HandleTypingUpdateAsync(ConnectedClient client, ClientSession session, NetworkMessage message, CancellationToken ct)
277 {
278     var update = message.GetPayload<TypingUpdatePayload>();
279     if (update == null)
280     {
281         await session.SendErrorAsync(ErrorCode.InvalidMessage, "Invalid typing update payload.");
282         return;
283     }
284
285     if (client.CurrentRoomCode == null)
286     {
287         await session.SendErrorAsync(ErrorCode.NotInRoom, "You are not in a room.");
288         return;
289     }
290
291     var room = _serverState.GetRoom(client.CurrentRoomCode);
292     if (room == null)
293     {
294         await session.SendErrorAsync(ErrorCode.RoomNotFound, "Room not found.");
295         return;
296     }
297     if (room.Status != RoomStatus.Racing) return;
298
299     if (room.Players.TryGetValue(client.UserId, out var playerState))
300     {
301         if (playerState.IsFinished) return;
302
303         // Clamp các giá trị từ client - không tin client
304         var maxChars = room.PassageText?.Length ?? 0;
305         var typedText = NormalizeTypedText(update.TypedText);
306         var typedAnalysis = string.IsNullOrEmpty(typedText)
307             ? AnalyzeClientCounters(update.CurrentPosition, update.CorrectChars, update.WrongChars, maxChars)
308             : AnalyzeTypedText(room.PassageText ?? string.Empty, typedText);
309
310         playerState.CurrentPosition = typedAnalysis.CurrentPosition;
311         playerState.CorrectChars = typedAnalysis.CorrectChars;
312         playerState.WrongChars = typedAnalysis.WrongChars;
313         playerState.BestStreak = Math.Max(playerState.BestStreak, typedAnalysis.BestStreak);
314         playerState.ObserveMistakes(room.PassageText ?? string.Empty, typedText);
315         if (!string.IsNullOrEmpty(typedText))
316             room.SetPlayerTypedText(client.UserId, typedText);
317
318         // Calculate WPM (dùng giá trị đã clamp)
319         if (room.StartedAt.HasValue)
320         {

```

Các bước xử lý chính (Clamp):

- normalize text: (305)
- phân tích lại thay vì tin client: (306)
- ghi nhận lỗi realtime: (314)
- lưu typed text mới nhất: (316)

1.2 Dữ liệu lỗi raw dùng cho AI có thật không?

Hệ thống không chỉ dùng report cuối race, mà lấy cả lỗi quan sát trong lúc gõ:

src/TypeRacer.Server/State/GameRoom.cs

```
145 public void ObserveMistakes(string passageText, string typedText)
146 {
147     if (string.IsNullOrEmpty(passageText) || string.IsNullOrEmpty(typedText))
148         return;
149
150     lock (_mistakeLock)
151     {
152         var max = typedText.Length;
153         for (var i = 0; i < max; i++)
154         {
155             var expected = i < passageText.Length ? passageText[i] : '\0';
156             var actual = i < typedText.Length ? typedText[i] : '\0';
157             if (expected == actual)
158                 continue;
159
160             var key = $"{i}:{expected}:{actual}";
161             if (!_observedMistakeKeys.Add(key))
162                 continue;
163
164             AddMistakeCharacter(expected);
165             AddMistakeCharacter(actual);
166
167             var expectedWord = ExtractWordAt(passageText, i);
168             if (!string.IsNullOrEmpty(expectedWord))
169                 Increment(_observedMistakeWords, expectedWord);
170
171             foreach (var ngram in ExtractNgramsAt(passageText, i))
172                 Increment(_observedMistakeNgrams, ngram);
173         }
174     }
175 }
```

Ở đây server bắt:

- ký tự sai
- từ sai
- n-gram sai

Cuối race, server lưu vào volatile memory cho AI:

src/TypeRacer.Server/Services/MistakeMemoryService.cs

```

14 public void RecordRaceResults(
15     string roomCode,
16     int raceId,
17     string passageText,
18     string gameMode,
19     IEnumerable<RaceResultDto> results,
20     DateTime playedAt)
21 {
22     PurgeExpired();
23
24     var normalizedRoom = NormalizeRoomCode(roomCode);
25     if (string.IsNullOrEmpty(normalizedRoom) || raceId <= 0)
26         return;
27
28     var safePassage = Truncate(passageText, MaxPassageTextLength);
29     foreach (var result in results.Where(r => !r.IsAiBot && r.UserId > 0))
30     {
31         var typedText = Truncate(result.TypedText, MaxTypedTextLength);
32         var observedChars = TrimCounts(result.ObservedMistakeCharacters, 12);
33         var observedWords = TrimCounts(result.ObservedMistakeWords, 12);
34         var observedNgrams = TrimCounts(result.ObservedMistakeNgrams, 16);
35         if (string.IsNullOrEmpty(typedText) &&
36             observedChars.Count == 0 &&
37             observedWords.Count == 0 &&
38             observedNgrams.Count == 0)
39         {
40             continue;
41         }
42
43         var key = BuildKey(normalizedRoom, raceId, result.UserId);
44         _samples[key] = new MistakeMemorySample
45         {
46             RaceId = raceId,
47             RoomCode = normalizedRoom,
48             UserId = result.UserId,
49             Username = result.Username,
50             PassageText = safePassage,
51             TypedText = typedText,
52             GameMode = gameMode,
53             PlayedAt = playedAt,
54             ObservedMistakeCharacters = observedChars,
55             ObservedMistakeWords = observedWords,
56             ObservedMistakeNgrams = observedNgrams,
57         };
58     }
59
60     TrimIfNeeded();

```

Các field lưu gồm:

- PassageText
- TypedText
- ObservedMistakeCharacters
- ObservedMistakeWords
- ObservedMistakeNgrams

1.3 AI lấy dữ liệu này như thế nào

Stats handler lấy sample lỗi rồi truyền vào AI. Prompt AI được build từ đúng dữ liệu đó:

src/TypeRacer.Server/Services/AiCoachService.cs

```

644 private static string BuildPrompt(
645     string username,
646     GetAiCoachRequest request,
647     CoachAnalysis analysis,
648     IReadOnlyList<MistakeMemorySample> mistakeSamples,
649     int attempt)
650 {
651     var passageSample = string.IsNullOrEmpty(request.PassageText)
652         ? "(không có)"
653         : BuildPassageSample(request.PassageText);
654     var topMistypedCharacters = analysis.TopMistypedCharacters.Count == 0
655         ? "(chưa xác định)"
656         : string.Join(", ", analysis.TopMistypedCharacters);
657     var topMistypedWords = analysis.TopMistypedWords.Count == 0
658         ? "(chưa xác định)"
659         : string.Join(", ", analysis.TopMistypedWords);
660     var topMistypedNgrams = analysis.TopMistypedNgrams.Count == 0
661         ? "(chưa xác định)"
662         : string.Join(", ", analysis.TopMistypedNgrams);
663
664     var completionPercent = analysis.CompletionRate * 100m;
665     var observedSummary = BuildObservedMistakeSummary(mistakeSamples);
666
667     return
668         $"Người chơi: {username}\n" +
669         $"RaceId: {request.RaceId}, Room: {request.RoomCode}\n" +
670         $"WPM: {request.Wpm:F1}, Accuracy: {request.Accuracy:F1}%\n" +
671         $"Position: {request.Position}/{request.TotalPlayers}, Completed: {request.IsCompleted}\n" +
672         $"Correct: {request.CharsCorrect}, Wrong: {request.CharsWrong}, TimeMs: {request.TimeTakenMs}, Completion: {completionPercent:F1}%\n" +
673         $"Recent races: count={request.RecentRaceCount}, completed={request.RecentCompletedCount}, avg_wpm={request.RecentAvgWpm:F1}, avg_acc={request.RecentAvgAccuracy:F1}%\n" +
674         $"Mật độ lỗi ký tự: {analysis.MistakeDensity * 100m:F1}% | Ký tự hay sai: {topMistypedCharacters}\n" +
675         $"Từ hay sai: {topMistypedWords}\n" +
676         $"N-gram/bigram/trigram hay sai: {topMistypedNgrams}\n" +
677         $"Ngôn ngữ: {NormalizeLanguage(request.Language)}\n" +
678         $"Passage sample: {passageSample}\n" +
679         $"Observed mistake samples in volatile memory: {mistakeSamples.Count}\n" +
680         $"Observed mistakes from live typing updates (includes mistakes later corrected):\n{observedSummary}\n" +
681         $"Phân tích heuristic hiện tại: focus={analysis.FocusArea}, tier={analysis.SkillTier}, estimated_next_wpm={analysis.EstimatedNextWpm:F1}\n" +
682         $"Return compact strict JSON with this exact shape and no markdown:\n" +
683         "{\n" +
684         "  \"coach_text\": \"1-2 concise sentences\",\n" +
685         "  \"training_title\": \"short title for the next training pack\",\n" +
686         "  \"recommended_game_mode\": \"classic|accuracy|no_backspace|sudden_death|ai_practice\",\n" +
687         "  \"recommended_difficulty\": \"easy|medium|hard|nightmare\",\n" +
688         "  \"recommended_target_rpm\": 45,\n"

```

Phần chứng minh AI dựa vào lỗi thật:

- top mistake chars (dòng 654)
- top mistake words (dòng 657)
- top n-grams (dòng 660)
- observed summary từ volatile memory (dòng 665)

2. Làm việc với dữ liệu nhập xuất

2.1 Dùng I/O stream gì

Project dùng:

- `TcpClient` / `TcpListener`
- `NetworkStream`
- `ReadAsync` / `WriteAsync`

Code chính:

src/TypeRacer.Client/Network/TcpGameClient.cs

```

27 public async Task ConnectAsync(string host, int port)
28 {
29     // Dọn dẹp kết nối cũ nếu còn
30     Disconnect();
31
32     _tcpClient = new TcpClient();
33
34     // Timeout 10 giây cho kết nối
35     using var timeoutCts = new CancellationTokSource(TimeSpan.FromSeconds(10));
36     await _tcpClient.ConnectAsync(host, port, timeoutCts.Token);
37     _stream = _tcpClient.GetStream();
38
39     // Bắt đầu vòng lặp nhận message
40     _cts = new CancellationTokSource();
41     _ = Task.Run(() => ReceiveLoopAsync(_cts.Token));
42 }
43
44 /// <summary>Gửi message tới server (thread-safe)</summary>
45 public async Task SendAsync(NetworkMessage message)
46 {
47     if (_stream == null) throw new InvalidOperationException("Chưa kết nối tới server.");
48
49     await _sendLock.WaitAsync();
50     try
51     {
52         var outbound = EncryptPayloads && message.Body.Length > 0
53             ? new NetworkMessage
54             {
55                 Type = message.Type,
56                 Flags = message.Flags | MessageFlags.Encrypted,
57                 Body = message.Body,
58             }
59             : message;
60
61         await MessageSerializer.WriteAsync(_stream, outbound);
62     }
63     finally
64     {
65         _sendLock.Release();
66     }
67 }

```

- client connect lấy stream:
- client gửi message
- server gửi message: (src/TypeRacer.Server/Network/ClientSession.cs:19)
- serializer ghi header + body (src/TypeRacer.Shared/Protocol/MessageSerializer.cs:12)
- reader đọc đủ byte từ TCP stream: (src/TypeRacer.Shared/Protocol/MessageReader.cs:11)

3. Mã hóa

3.1 Code mã hóa pass đăng kí/đăng nhập

src\TypeRacer.Server\Services\AuthService.cs

```

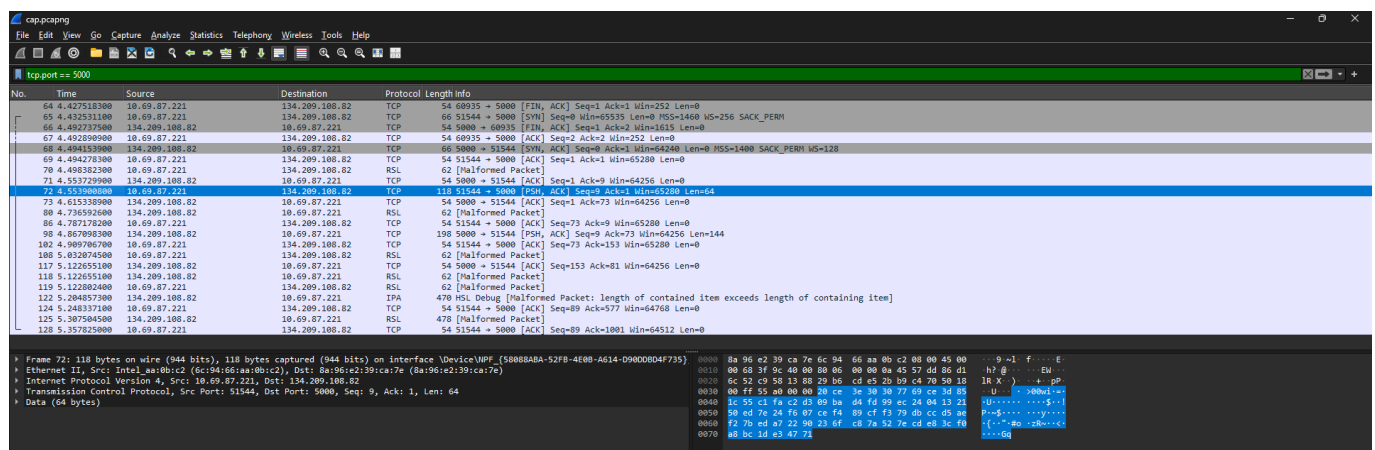
60 public async Task<RegisterResponse> RegisterAsync(string username, string password)
61 {
62     if (string.IsNullOrEmpty(username) || username.Length < 3 || username.Length > 50)
63         return new RegisterResponse { Success = false, ErrorMessage = "Tên đăng nhập phải từ 3-50 ký tự." };
64
65     // Chỉ cho phép chữ cái, số, gạch dưới, gạch ngang
66     if (!System.Text.RegularExpressions.Regex.IsMatch(username, @"^[a-zA-Z0-9_\-]+$"))
67         return new RegisterResponse { Success = false, ErrorMessage = "Tên đăng nhập chỉ chứa chữ cái, số, _ và -." };
68
69     if (string.IsNullOrEmpty(password) || password.Length < 6)
70         return new RegisterResponse { Success = false, ErrorMessage = "Mật khẩu tối thiểu 6 ký tự." };
71
72     if (await _userRepo.UsernameExistsAsync(username))
73         return new RegisterResponse { Success = false, ErrorMessage = "Tên đăng nhập đã được sử dụng." };
74
75     var hash = HashHelper.HashPasswordV3(password);
76     try
77     {
78         await _userRepo.CreateAsync(username, hash);
79     }
80     catch (DuplicateUsernameException)
81     {
82         return new RegisterResponse { Success = false, ErrorMessage = "Tên đăng nhập đã được sử dụng." };
83     }
84
85     return new RegisterResponse { Success = true };
86 }
87 }

```

Auth service:

- verify password
- tạo session token
- hash password khi register
- HashHelper.HashPasswordV3() trong c# dùng PBKDF2-HMAC-SHA256 + salt

3.2 Bắt Wireshark đăng kí/dăng nhập xem có mã hóa không



- Luồng đăng nhập là TCP stream 7: 10.69.87.221:51544 -> 134.209.108.82:5000
- Protocol của app dùng header 8 byte: [BodyLength:4][Type:2][Flags:2]

1. Frame 70 là header của LOGIN_REQUEST

- payload 8 byte: 00 00 00 40 00 64 00 01
- giải mã:

- BodyLength = 0x00000040 = 64
- Type = 0x0064 = 100 = LOGIN_REQUEST
- Flags = 0x0001 = Encrypted

2. Frame 72 là body của LOGIN_REQUEST dài đúng 64 bytes

```
20 ce 3e 30 30 77 69 ce 3d 85 1c 55 c1 fa c2 d3
09 ba d4 fd 99 ec 24 04 13 21 50 ed 7e 24 f6 07
ce f4 89 cf f3 79 db cc d5 ae f2 7b ed a7 22 90
23 6f c8 7a 52 7e cd e8 3c f0 a8 bc 1d e3 47 71
```

- phần này là ciphertext, không đọc được username/password dạng text hay JSON.

Kết luận: Body 64 byte ở Frame 72 là dữ liệu nhị phân ngẫu nhiên, không lộ username/password dưới dạng plain text.

3.3 Code mã hóa chat

- Client bật flag mã hóa khi gửi
- Server bật flag mã hóa khi trả response
- Serializer mã hóa body nếu có flag: src/TypeRacer.Shared/Protocol/MessageSerializer.cs

```
10 public static class MessageSerializer
11 {
12     public static async Task WriteAsync(Stream stream, NetworkMessage message, CancellationToken ct = default)
13     {
14         var body = message.Body;
15         var flags = message.Flags;
16
17         // Encrypt body if flag is set
18         if (flags.HasFlag(MessageFlags.Encrypted) && body.Length > 0)
19         {
20             body = AesEncryption.Encrypt(body);
21         }
22
23         // Build header: [BodyLength:4][Type:2][Flags:2]
24         var header = new byte[Constants.HeaderSize];
25         BinaryPrimitives.WriteInt32BigEndian(header.AsSpan(0, 4), body.Length);
26         BinaryPrimitives.WriteUInt16BigEndian(header.AsSpan(4, 2), (ushort)message.Type);
27         BinaryPrimitives.WriteUInt16BigEndian(header.AsSpan(6, 2), (ushort)flags);
28
29         // Write header + body
30         await stream.WriteAsync(header, ct);
31         if (body.Length > 0)
32         {
33             await stream.WriteAsync(body, ct);
34         }
35         await stream.FlushAsync(ct);
36     }
37 }
```


- Reader giải mã khi nhận: src/TypeRacer.Shared/Protocol/MessageReader.cs

```

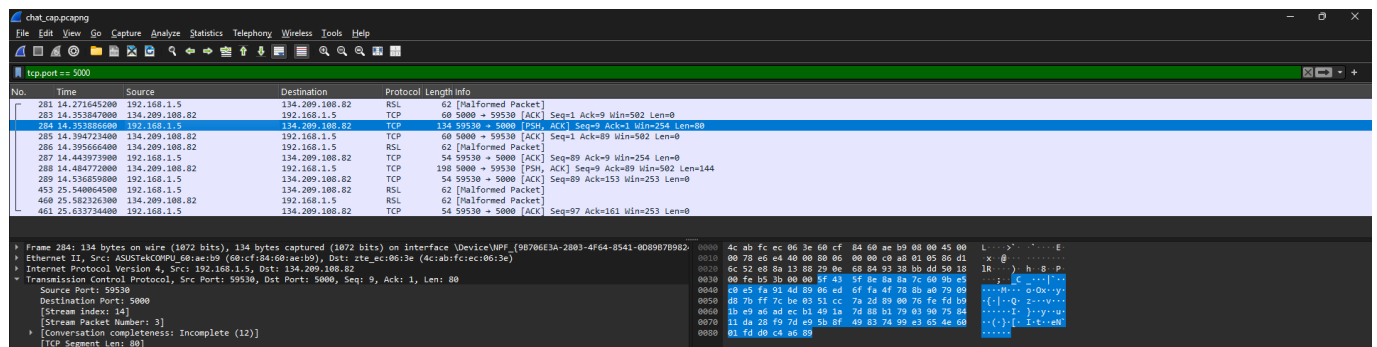
26 // Read body
27 var body = Array.Empty<byte>();
28 if (bodyLength > 0)
29 {
30     body = new byte[bodyLength];
31     if (!await ReadExactAsync(stream, body, ct))
32         return null; // Connection closed mid-message
33
34     // Decrypt if encrypted
35     if (flags.HasFlag(MessageFlags.Encrypted))
36     {
37         body = AesEncryption.Decrypt(body);
38     }
39 }
40

```

Thuật toán:

- AES-256-CBC
- random IV mỗi message

3.4 Bắt Wireshark việc gửi chat xem có mã hóa không



Luồng chính:

- 192.168.1.5:59530 -> 134.209.108.82:5000

CHAT_SEND

Frame 281 là header: 00 00 00 50 01 90 00 01

Giải mã:

- BodyLength = 80
- Type = 0x0190 = 400 = CHAT_SEND
- Flags = 0x0001 = Encrypted

Frame 284 là body 80 byte đã mã hóa:

```

5f435f8e8a8a7c609be5c0e5fa914d8906ed6ffa4f788ba07909d87bfff7cbe0351cc7a2d890076fefdb91be9a6adecb1491a7d88b1790390758411da28f97de95b8f49837499e3654e6001fdd0c4a689

```

Kết luận: Không đọc được nội dung chat ở dạng text thường.

4. Demo mạng LAN

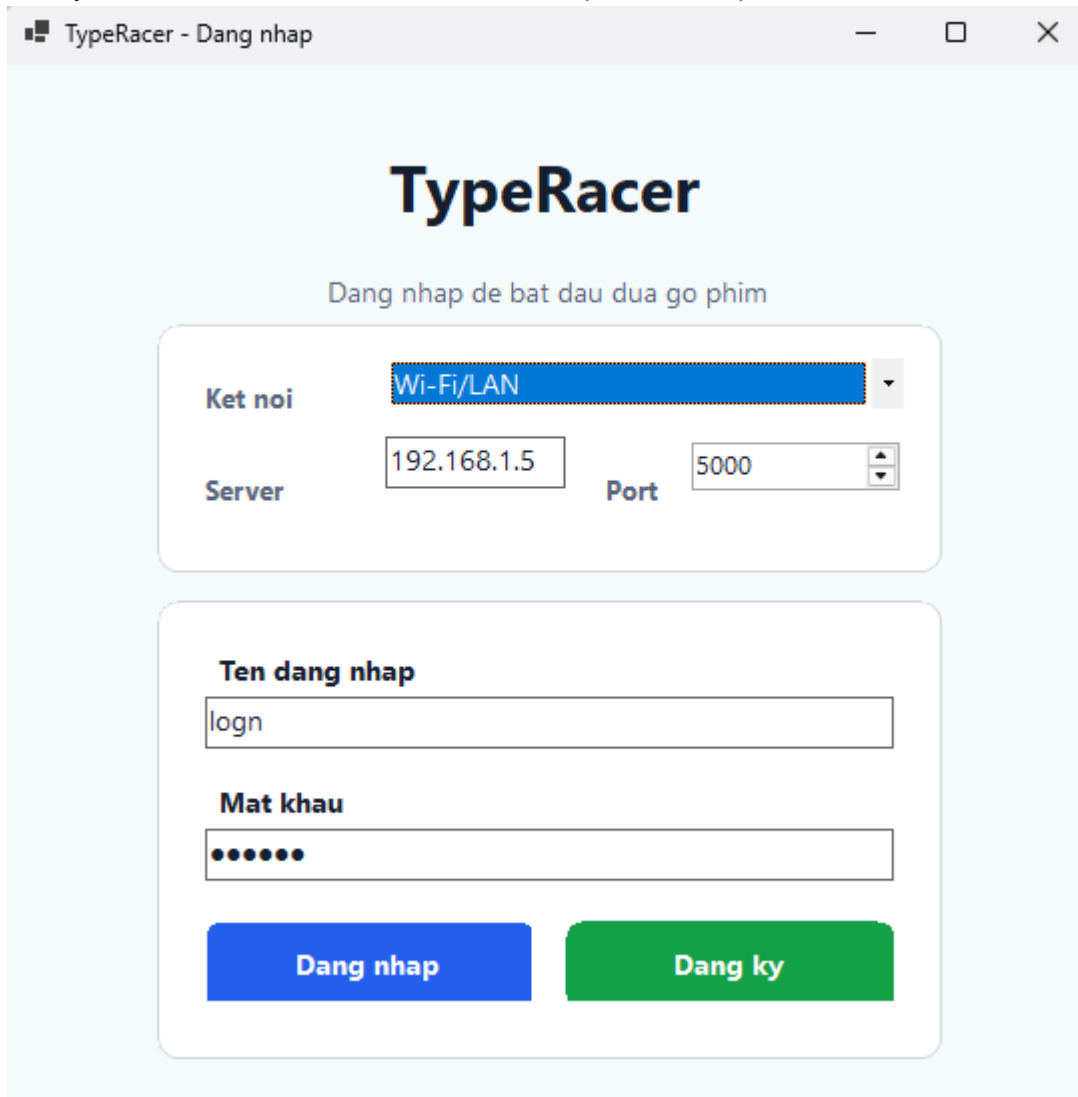
4.1 Cách chứng minh đang kết nối qua LAN

Host trên một máy tính 1:

IP private nội bộ lấy từ ipconfig **192.168.1.5:5000**

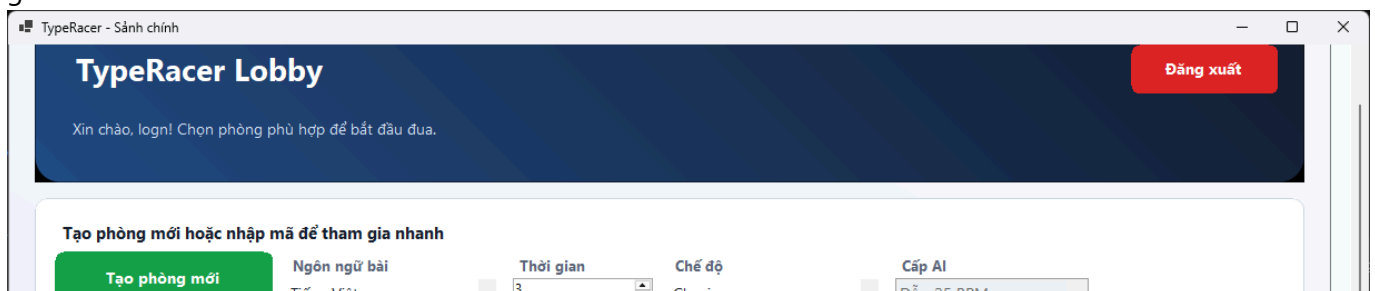
```
Link-local IPv6 Address . . . . . : fe80::82e8:70ac:1005:ad4b%1
IPv4 Address. . . . . : 192.168.1.5
Subnet Mask . . . . . : 255.255.255.0
```

Ở máy 2 làm client, chọn chế độ Wifi/LAN, nhập chính xác ip trên



Kết quả là vào được

game



Kiểm tra trong cmd cho thấy server local đang listen ở máy host.

```
C:\Users\ADMIN>netstat -ano | findstr LISTENING
```

TCP	0.0.0.0:135	0.0.0.0:0	LISTENING	1812
TCP	0.0.0.0:445	0.0.0.0:0	LISTENING	4
TCP	0.0.0.0:3306	0.0.0.0:0	LISTENING	9120
TCP	0.0.0.0:5000	0.0.0.0:0	LISTENING	37224
TCP	0.0.0.0:5040	0.0.0.0:0	LISTENING	11396
TCP	0.0.0.0:5432	0.0.0.0:0	LISTENING	8412